

BSc (Hons) Computing Systems

COM661 (18368) - Full-stack Strategies and Development

Api-Endpoint Summary

Author

B00957996 : Md Nasif

Emplivio- Employee Management Application

1. Dataset Overview

The Employee Management Application is designed to store and manage essential information about employees within an organisation. The dataset provides the information required to manage employees, departments, job roles, attendance, and leave records.

The application will allow authorised users to:

- Add and manage employee records
- View and update employee information
- Organise employees by department and job role
- Record employee attendance
- Manage employee leave requests
- Search and filter employee records

The dataset is organised into separate collections of related information so that employee records can be managed efficiently and expanded when required.

2. Dataset Structure

The dataset consists of several main entities. Each entity stores information for a specific part of the employee management system.

2.1 User

Field	Description
user_id	Unique user identifier
username	User's username
email	User's email address
password	Encrypted password
admin	Admin access
role	User access role
created_at	Account creation date-time
updated_at	Account update date-time
Account_status	States an account status

2.2 Employee

Field	Description
employee_id	Unique employee identifier
personal_details	**Embedded document containing the employee's personal information**
personal_details.first_name	Employee's first name
personal_details.last_name	Employee's last name
personal_details.date_of_birth	Employee's date of birth
personal_details.contact	**Embedded document containing contact information**
personal_details.contact.email	Employee's email address

personal_details.contact.phone	Employee's contact number
personal_details.contact.address	**Embedded document containing the employee's address**
personal_details.contact.address.street	Employee's street address
personal_details.contact.address.city	Employee's city
personal_details.contact.address.postcode	Employee's postal code
personal_details.contact.address.country	Employee's country
department_id	Reference identifier for the employee's department
role_id	Reference identifier for the employee's job role
status	Employee's current employment status (e.g. Active or Inactive)
employment	**Embedded document containing employment information**
employment.hire_date	Date the employee joined the organisation
employment.employment_type	Type of employment (e.g. Permanent, Fixed-term or Part-time)
employment.compensation	**Embedded document containing employee compensation information**
employment.compensation.current_salary	Employee's current salary
employment.compensation.currency	Currency used for the employee's salary
employment.compensation.salary_reviews	**Array containing the employee's salary review history**
employment.compensation.salary_reviews.review_id	Unique identifier for each salary review
employment.compensation.salary_reviews.previous_salary	Employee's salary before the review
employment.compensation.salary_reviews.new_salary	Employee's salary after the review
employment.compensation.salary_reviews.reason	Reason for the salary adjustment

2.3 Department

Field	Description
department_id	Unique department identifier
name	Department name
description	Brief department description
manager_id	Employee responsible for the department

2.4 Job Role

Field	Description
role_id	Unique role identifier
name	Job role name
description	Brief description of the role

Salary_range	**Embedded document containing salary information**
Salary_range.minimum	Minimum salary
Salary_range.maximum	Maximum salary

2.5 Attendance

Field	**Description**
attendance_id	Unique attendance record identifier
employee_id	Employee associated with the record
date	Attendance date
status	Present, Absent or Late
check_in	Employee check-in time
check_out	Employee check-out time

2.6 Leave Request

Field	**Description**
leave_id	Unique leave request identifier
employee_id	Employee making the request
leave_type	Type of leave
Period	**Embedded document containing leave date information**
Period.start_date	Leave start date
Period.end_date	Leave end date
reason	Reason for the leave
status	Pending, Approved or Rejected
requested_at	Date the request was submitted

3. Application Domain

Emplivio is an Employee Management System within the Human Resources (HR) and Workforce Management domain. It provides a centralised platform for managing employee information and supporting essential organisational activities, including employee records, department and job role management, attendance tracking, leave management and employee administration.

Main Users

Administrator – Manages users, employees, departments, job roles and overall system information.

HR Staff – Manages employee information, attendance records and leave requests.

Manager – Views employees within their department and reviews relevant attendance and leave information.

Employee- Manages own account and information.

Main Application Functions

- Employee management
- Department management
- Job role management
- Attendance management
- Leave management
- User authentication
- Basic employee reports and summaries

4. Implementation Plan

The application will be developed using a frontend, backend API and database. The backend will provide a set of REST API endpoints for managing the main application functions.

API Endpoints

Authentication

- **GET /**– Welcomes a user.
- **POST /register** – Create a new user account.
- **POST /login** – Authenticate a user and generate an access token.

Employee Management

- **GET /employees/<employee_id>/profile**– Retrieve an employe complete profile.
- **GET /employees** – Retrieve all employees with optional filtering and pagination.
- **POST /employees** – Create a new employee record.
- **GET /employees/<employee_id>** – Retrieve information about a specific employee.
- **PUT /employees/<employee_id>** – Update employee information.
- **DELETE /employees/<employee_id>** – Remove an employee record (Admin only).

Department Management

- **POST /departments** – Create a new department.
- **GET /departments** – Retrieve all departments.
- **GET /departments/<department_id>** – Retrieve a specific department.
- **PUT /departments/<department_id>** – Update department information.
- **DELETE /departments/<department_id>** – Remove a department (Admin only).

Job Role Management

- **POST /roles** – Create a new job role.
- **GET /roles** – Retrieve all available job roles.
- **GET /roles/<role_id>** – Retrieve a specific job role.
- **PUT /roles/<role_id>** – Update job role information.
- **DELETE /roles/<role_id>** – Remove a job role (Admin only).

Attendance Management

- **GET / attendances** – Retrieve attendance records for all employees.
- **GET /attendances/< attendance_id>** – Retrieve attendance records for an employee.
- **POST /attendances** – Add an attendance record for an employee.

- **PUT /attendances/<attendance_id>** – Update an attendance record.
- **DELETE /attendances/<attendance_id>** – Delete an attendance record.

Leave Management

- **GET /leaves** – Retrieve all employee's leave requests.
- **GET /leaves/< leave_id >** – Retrieve an employee's leave requests.
- **POST /leaves** – Submit a new leave request.
- **PUT /leaves/<leave_id>** – Update a leave request or its status.
- **DELETE /leaves/<leave_id>** – Delete a leave request.

Admin Management

- **GET/admin/users - Retrieve all system users**
- **GET/admin/users/<user_id> - Retrieve a specific user**
- **PUT/admin/users/<user_id> - Update user details or role**
- **DELETE/admin/users/<user_id> - Remove a user account**
- **GET/admin/dashboard - Retrieve overall system summary**

Basic Analytics

- **GET /analytics/employee-count** – Calculate the total number of employees.
- **GET /analytics/department-summary** – Show the number of employees in each department.
- **GET /analytics/attendance-summary** – Provide a basic attendance summary.
- **GET /analytics/leave-summary** – Provide a summary of leave requests.

Implementation Flow

The frontend will send requests to the API. The backend will validate and process the requests, communicate with the database, and return the required information to the frontend.

Frontend → REST API → Backend → Database

For example, when a user requests GET /employees, the backend retrieves the employee records and returns them to the frontend. When a new employee is added, the frontend sends a POST request and the backend validates and stores the new record.

Authentication and Access

Users will be required to log in before accessing protected parts of the application. Access will depend on the user's role. Administrators will have full access to functions relevant to their responsibilities.

Testing

- Test API endpoints for correct responses.
- Check that employee information is stored and updated correctly.
- Test authentication and access permissions.
- Test employee, attendance and leave forms.
- Test communication between the frontend, API and database.